



[ChromiumOS > Guides](#) >

ChromiumOS Developer Guide

Introduction

This guide describes how to work on ChromiumOS. If you want to help develop ChromiumOS and you're looking for detailed information about how to get started, you're in the right place.

Contents

[Introduction](#)

[Target audience](#)

[Organization & content](#)

[Typography conventions](#)

[Modifying this document](#)

[More information](#)

[Prerequisites](#)

[Python](#)

[Install development tools](#)

[Install depot_tools](#)

[Tweak your sudoers configuration](#)

[Set locale](#)

[Configure git](#)

[Get credentials to access source repos](#)

[Double-check that you are running a 64-bit architecture](#)

[Verify that your default file permissions \(umask\) setting is correct](#)

[Get the Source](#)

[Decide where your source will live](#)

[Compatible Filesystems](#)

[Get the source code](#)

[Sync to stable](#)

[Enable cros cron for faster syncing and builds](#)

[Optionally add Google API keys](#)

[Branch Builds](#)

[Make sure you are authorized to access Google Storage \(GS\) buckets](#)

[Authenticate for remote Bazel caching with RBE, if applicable](#)

[Building ChromiumOS](#)

[Select a board](#)

[Build the packages for your board](#)

[Build a disk image for your board](#)

[Installing ChromiumOS on your Device](#)

[Put your image on a USB disk](#)

[Enter Developer Mode](#)

[Boot from your USB disk](#)

[Getting to a command prompt on ChromiumOS](#)

[Installing your ChromiumOS image to your hard disk](#)

[Running an image in a virtual machine](#)

[Making changes to packages whose source code is checked into ChromiumOS git repositories](#)

- Keep the tree green
- Run cros workon start
- Run repo sync
- Find out which ebuids map to which directories
- Create a branch for your changes
- Make your changes
- Set your editor
- Submit changes locally
- Upload your changes and get a code review
- Clean up after you're done with your changes
- Making changes to non-cros-workon-able packages
 - Adding small patches to existing packages
 - Making changes to the way that the chroot is constructed
 - Building an individual package
 - Making changes to the Chromium web browser on ChromiumOS
- Local Debugging
 - Debugging both x86 and non-x86 binaries on your workstation
 - Using the cros debug tool
 - Using gdb-`{BOARD}`
 - Printing stack traces at runtime
- Remote Debugging
 - Setting up remote debugging by hand
 - Automated remote debugging using cros debug
 - Setup notes:
 - cros debug command line options summary
 - Example: debugging a binary locally with cros debug
 - Example: debugging a running process on a remote device with cros debug
 - Tips For Googlers
 - Automated remote debugging using gdb-`{BOARD}` script (gdb-lumpy, gdb-daisy, gdb-parrot, etc)
 - Edit/Debug cycle
 - Examples of debugging using the gdb-`{BOARD}` script
- Troubleshooting
 - I lost my developer tools on the stateful partition, can I get them back?
 - Disabling Enterprise Enrollment
- Running Tests
 - Set up SSH connection between chroot and DUT
 - For Googlers
 - Tast
 - Autotest
 - Creating a normal image that has been modified for test
 - Creating a recovery image that has been modified for test
- Additional information
 - Entering a shell in the chroot
 - Inspecting a disk image
 - Working on SDK Packages
 - Documentation on this site
 - External documentation

Target audience

The target audience of this guide is anyone who wants to obtain, build, or contribute to ChromiumOS. That includes new developers who are interested in the project and who simply want to browse through the ChromiumOS code, as well as developers who have been working on ChromiumOS for a long time.

Organization & content

This guide describes the common tasks required to develop ChromiumOS. The guide is organized linearly, so that developers who are new to ChromiumOS can follow the tasks in sequence. The tasks are grouped into the following sections:

- [Prerequisites](#)
- [Getting the source code](#)
- [Building ChromiumOS](#)
- [Installing ChromiumOS on your Device](#)
- [Making changes to packages whose source code is checked into ChromiumOS git repositories](#)
- [Making changes to non-cros-workon-able packages](#)
- [Local Debugging](#)
- [Remote Debugging](#)
- [Troubleshooting](#)
- [Running Tests](#)
- [Additional information](#)

Typography conventions

- **Commands** are shown with different labels to indicate whether they apply to (1) your build computer (the computer on which you're doing development), or (2) your ChromiumOS computer (the device on which you run the images you build):

Label	Commands
(outside)	on your build computer, outside the chroot
(device)	on your ChromiumOS computer

Beneath the label, the command(s) you should type are prefixed with a generic shell prompt, `$` . This distinguishes input from the output of commands, which is not so prefixed.

- **Notes** are shown using the following conventions:
 - **IMPORTANT NOTE** describes required actions and critical information
 - **SIDE NOTE** describes explanations, related information, and alternative options
- **TODO** describes questions or work that is needed on this document

Modifying this document

If you're a ChromiumOS developer, **YOU SHOULD UPDATE THIS DOCUMENT** and fix things as appropriate. See [CONTRIBUTING.md](#) for how to update this document. Bias towards action:

- If you see a TODO and you know the right answer, fix it!
- If you see something wrong, fix it.
- If you're not sure of the perfect answer, still fix it. Stick in a TODO noting your uncertainty if you aren't sure, but don't let anything you know to be wrong stick around.

Please try to abide by the following guidelines when you modify this document:

- Put all general "getting started" information directly on this page. It's OK if this document grows big.

- If some information is also relevant for other documents, put the information in this document and add links from the other documents to this document. If you do not have a choice and must put information relevant to getting started in other documents, add links in this document to discrete chunks of information in the other documents. Following a web of links can be very confusing, especially for people who are new to a project, so it's very important to maintain a linear flow as much as possible.
- Where possible, describe a single way of doing things. If you specify multiple options, provide a clear explanation why someone following the instructions would pick one option over the others. If there is disagreement about how to do things, consider listing alternative options in a SIDE NOTE.
- Keep Google-specific references to a minimum. A large number of the people working on ChromiumOS work at Google, and this document may occasionally contain references that are only relevant to engineers at Google. Google engineers, please keep such references to a minimum – ChromiumOS is an open source project that should stay as open as possible.

More information

This document provides an overview of the tasks required to develop ChromiumOS. After you've learned the basics, check out the links in the [Additional information](#) section at the end of this document for tips and tricks, FAQs, and important details (e.g., the ChromiumOS directory structure, using the dev server, etc.).

Finally, if you build a ChromiumOS image, please read this important note about [Attribution requirements](#).

Prerequisites

You must have Linux to develop ChromiumOS. Any recent or up-to-date distribution should work. However, we can't support everyone and their dog's Linux distro, so the only official supported environment is listed below. If you encounter issues with other setups, patches are generally welcomed, but please do not expect us to figure out your distro.

- [Ubuntu](#) Linux (version 22.04 - Jammy)

Most developers working on ChromiumOS are using Jammy (the 22.04 LTS version of Ubuntu) and Debian testing (Trixie). Things might work if you're running a different Linux distribution, but you will probably find life easier if you're on one of these.

Ubuntu 24.04 has [disabled user namespaces](#) by default. Run the following to re-enable them, otherwise you might encounter some strange failures, such as `Permission denied: '/proc/self/setgroups':`

```
sudo systemctl -w kernel.apparmor_restrict_unprivileged_unconfined=0
sudo systemctl -w kernel.apparmor_restrict_unprivileged_usersns=0
```

- an x86_64 64-bit system for performing the build
- an account with `sudo` access

You need root access to run the `chroot` command and to modify the mount table. **NOTE:** Do not run any of the commands listed in this document as root – the commands themselves will run `sudo` to get root access when needed.

- many gigabytes of RAM

Per [this thread](#), linking Chrome requires somewhere between 8 GB and 28 GB of RAM as of March 2017; you may be able to get by with less at the cost of slower builds with adequate swap space. Seeing an error like `error: ld terminated with signal 9 [Killed]` while building the `chromeos-chrome` package indicates that you need more RAM. If you are not building your own copy of Chrome, the RAM requirements will be substantially lower.

You will have a much nicer time if you also have:

- a fast multi-processor machine with lots of memory

The build system is optimized to make good use of all processors, and an 8 core machine will build nearly 8x faster than a single core machine.

- a good Internet connection

This will help for the initial download (minimum of about 2 GB) and any further updates.

Python

If your system does not have a compatible Python version installed, you'll need to install Python 3.8 or greater.

To check your python version:

```
(outside)
$ python3 -V
```

If an error or a version lower than 3.8 is returned, proceed with the rest of the section. If not, skip this section.

If your system supports newer specific Python versions, you can install it. If an incompatible version is present, uninstall it:

```
(outside)
$ sudo apt-get remove <current Python package>
```

And install a new version:

```
(outside)
$ sudo apt-get install python3.9
```

Install development tools

Some host OS tools are needed to manipulate code, bootstrap the development environment, and run preupload hooks later on.

```
(outside)
# On Ubuntu, make sure to enable the universe repository.
$ sudo add-apt-repository universe
$ sudo apt-get install git gitk git-gui curl util-linux util-linux-extra
```

These commands also installs git's graphical front end ([git gui](#)) and revision history browser ([gitk](#)).

Install depot_tools

To get started, follow the initial instructions at [install depot_tools](#). You only need to clone the repo & setup your PATH - the rest of the document is for browser developers.

This step is required so that you can use the [repo](#) to get/sync the source code, as well as other CrOS specific tools.

Tweak your sudoers configuration

You can tweak your sudoers configuration to make `sudo` request your password less frequently as described in [Making sudo a little more permissive](#).

Set locale

These may not be needed if you are building on a system that you already use, however if you have a clean instance on GCE, you'll need to set a better locale. For example, on Debian on GCE, do:

```
(outside)
$ sudo apt-get install locales
$ sudo dpkg-reconfigure locales
```

When running `dpkg-reconfigure locales`, choose a language with UTF-8, e.g. `en_US.UTF-8`. For this change to take effect, you will need to log out and back in (closing all terminal windows, tmux/screen sessions, etc.).

Configure git

Setup git now. If you don't do this, you may run into errors/issues later. Replace `you@example.com` and `Your Name` with your information:

```
(outside)
$ git config --global user.email "you@example.com"
$ git config --global user.name "Your Name"
```

Get credentials to access source repos

Follow the [Gerrit guide](#) to get machine access credentials for the source repos.

This will also set up your code review account(s), which you can use to upstream changes back to ChromiumOS. This will be discussed in more detail in the "[Making changes to packages whose source code is checked into ChromiumOS git repositories](#)" section.

Double-check that you are running a 64-bit architecture

Run the following command:

```
(outside)
$ uname -m
```

You should see the result: `x86_64`

If you see something else (for example, `i686`, which means you are on a 32-bit machine or a 64-bit machine running a 32-bit OS) then you won't be able to build ChromiumOS. The project would happily welcome patches to fix this.

Verify that your default file permissions (umask) setting is correct

Sources need to be world-readable to properly function inside the chroot (described later). For that reason, the last digit of your umask should not be higher than 2, e.g. 002 or 022. Many distros have this by default; Ubuntu, for instance, does not. It is essential to put the following line into your `~/.bashrc` file before you checkout or sync your sources.

```
(outside)
$ umask 022
```

You can verify that this works by creating any file and checking if its permissions are correct.

```
(outside)
$ touch ~/foo
$ ls -la ~/foo
-rw-r--r-- 1 user group 0 2012-08-30 23:09 /home/user/foo
```

Get the Source

Decide where your source will live

ChromiumOS developers commonly put their source code in `~/chromiumos`. If you feel strongly, put your own source elsewhere, but note that **all commands in this document assume that your source code is in `~/chromiumos`**.

Create the directory for your source code with this command:

```
(outside)
$ mkdir -p ~/chromiumos
```

IMPORTANT NOTE: Please ensure the backing filesystem for the ChromiumOS code is compatible, as detailed in the next section.

Compatible Filesystems

Some filesystems are not compatible with the ChromiumOS build system, and as such, the code (and `chroot` and `out` directory) should not be hosted on such filesystems. Below lists the known compatibility issues:

Filesystem	Compatible
ext4	Yes
nfs	No
btrfs	Yes, but only without compression or other features

- nfs is not compatible because builds use `sudo` and root doesn't have access to nfs mount, unless your nfs server has the `no_root_squash` option. Furthermore, it is also a bad idea due to performance reasons.
- btrfs generally works, but any features that introduce xattr on files may cause issues with squashfs/erofs operations during the build process, and as such, should be avoided. Particularly, the btrfs compression feature is known to introduce xattr and cause build failures.

If your home directory is not backed by a compatible filesystem, then wherever you place your source, you can still add a symbolic link to it from your home directory (this is suggested), like so:

```
(outside)
$ mkdir -p /usr/local/path/to/source/chromiumos
$ ln -s /usr/local/path/to/source/chromiumos ~/chromiumos
```

Get the source code

ChromiumOS uses [repo](#) to sync down source code. [repo](#) is a wrapper for the [git](#) that helps deal with a large number of [git](#) repositories. You already installed [repo](#) when you [installed depot tools above](#).

The `repo init` command below uses a feature known as [sync to stable](#); the version of the source that you will sync to is 4-10 hours old but is verified by our CI system.

Public:

```
(outside)
$ cd ~/chromiumos
$ repo init -u https://chromium.googlesource.com/chromiumos/manifest -b stable
$ repo sync -j4
```

*** note **Note:** If you are using public manifest with devices that have restricted binary prebuilts, such as GPU drivers on ARM devices, you'll have to explicitly accept licenses. Read more at <https://www.chromium.org/chromium-os/licensing/building-a-distro/>

Googlers/internal manifest:

Gerrit Review-Enforcement has been enabled on the Chromium/Chrome gerrit hosts. In order to use the internal manifest and to only require one CR+2 from another Googler you need to use the `sso://` URLs when uploading a CL.

Add the following to your `.git/config`:

```
[url "sso://chromium"]
  insteadof = https://chromium.googlesource.com
  insteadof = https://chromium-review.googlesource.com
[url "sso://chrome-internal"]
  insteadof = https://chrome-internal.googlesource.com
  insteadof = https://chrome-internal-review.googlesource.com
```

Then run

```
(outside)
$ cd ~/chromiumos
$ repo init -u https://chrome-internal.googlesource.com/chromeos/manifest-internal -b stable
$ repo sync -j4
```

*** note **Note:** `-j4` tells `repo` to concurrently sync up to 4 repositories at once. You can adjust the number based on how fast your internet connection is. For the initial sync, it's generally requested that you use no more than 8 concurrent jobs. (For later syncs, when you already have the majority of the source local, using `-j16` or so is generally okay.)

*** note **Note:** If you are on a slow network connection or have low disk space, you can use the `-g minilayout` option. This starts you out with a minimum amount of source code. This isn't a particularly well tested configuration and has been known to break from time-to-time, so we usually recommend against it.

Sync to stable

As of October 2023, we recommend using `stable` (instead of `tip-of-tree/ToT/main`) for development. The `stable` ref points to a version of the source that is generally 4-10 hours old compared to ToT but is verified by our CI system.

If you have previously run `repo init` without the `-b stable`, you can convert an existing checkout to sync to stable:

```
(outside)
$ repo init -b stable
$ repo sync
```

The `stable` ref updates anytime snapshot-orchestrator passes all build and unittest stages for all build targets (it ignores VM/hardware tests so that `stable` is updated more frequently). `stable` will not update anytime ToT is broken, so if ToT is broken for a long period of time, `stable` could be much older than 4-10 hours.

*** note **Note:** Run `repo init -b main` to switch back to ToT. You would want to do this if you want to see a change that just landed on ToT and don't want to wait for the stable ref to be updated to include the change you are interested in building off of.

*** note **Note:** You can also sync to a specific version, for example, `R117-15550`. Please refer to [Switch repo to the snapshot](#).

Enable `cross cron` for faster syncing and builds

If you work on ChromiumOS often and want syncing your tree to be fast, turn on `cross cron` to automatically prefetch git objects and SDK artifacts hourly in the background:

```
(outside)
$ cross cron enable
```

*** note **Note:** `cross cron` daily bandwidth consumption of 10 GB. If you pay for your internet by the gigabyte, enabling it is not recommended.

*** note **Note:** `cross cron` works best when you ran `repo init` with either `-b stable` or `-b snapshot`. Using `-b main` will require a `git fetch` on every repository when you sync, and is therefore not recommended.

Optionally add Google API keys

*** note **Note:** If you plan to build the image with Chrome, you can skip this step. These keys are only necessary when you build the image with Chromium and need to use features that access Google APIs (**signing in**, translating web pages, geolocation, etc).

You will need to have keys (see [API Keys](#)) either in your `include.gypi`, or in a file in your home directory called `".googleapikeys"`. If either of these file are present for step 1 of building (below) they will be included automatically. If you don't have these keys, these features of Chromium will be quietly disabled.

Branch Builds

If you want to build on a branch, pass the branch name to `repo init` (e.g: `repo init -u <URL> [-g minilayout] -b release-R80-12739.B`).

When you use `repo init` you will be asked to confirm your name, email address, and whether you want color in your terminal. This command runs quickly. The `repo sync` command takes a lot longer.

More info can be found in the [working on a branch page](#).

Make sure you are authorized to access Google Storage (GS) buckets

Building and testing ChromiumOS requires access to Google Storage. This is done via [gsutil](#). Once configured, an authorization key is placed in `~/.boto`. Every time you access the [chroot](#) via `cros_sdk`, the `.boto` file is copied to the [chroot](#). If you run [gsutil](#) inside the chroot, it will configure the key in the chroot version of `~/.boto`, but every time you re-run `cros_sdk`, it will overwrite the `~/.boto` file in the chroot.

Authenticate for remote Bazel caching with RBE, if applicable

Some ChromiumOS packages use Bazel to build, which can interact with a remote cache such as RBE for faster builds. If your organization (Google, and maybe eventually others) has enabled remote caching in RBE for your Bazel-based builds, you'll need to install the [gcloud](#) CLI (<https://cloud.google.com/sdk/docs/install>, or [go/installgcloud](#) for Googlers) and run:

```
(outside)
$ gcloud auth application-default login
$ gcloud auth application-default set-quota-project chromeos-bot
```

This will store authentication credentials in `~/${HOME}/.config/gcloud` that will be mapped into the chroot, and which will allow Bazel executions within your builds to authenticate as you to RBE to access the remote cache it hosts.

If you need to disable remote caching for packages that build under Bazel and have remote caching enabled, this can be done via `BAZEL_USE_REMOTE_CACHING=false cros build-packages ...`

For Googlers, please reach out to chromeos-build-discuss@google.com if you believe you should have access to remote Bazel caching but don't.

More details about configuring remote caching for non-Google organizations is available at [bazel_remote_caching](#).

Building ChromiumOS

Select a board

Building ChromiumOS produces a disk image (usually just called an "image") that can be copied directly onto the boot disk of a computer intended to run ChromiumOS. Depending on the specifics of that computer, you may want different files in the disk image. For example, if your computer has an ARM processor, you'll want to make sure that all executables in the image are compiled for the ARM instruction set. Similarly, if your computer has special hardware, you'll want to include a matching set of device drivers.

Different classes of computers are referred to by ChromiumOS as different target "boards". The following are some example boards:

- **amd64-generic** - builds a generic image suitable for computers with an x86_64-compatible CPU (64 bit) or for running in a VM
- **arm-generic** - builds a generic image suitable for computers with an ARM CPU (32 bit)
- **arm64-generic** - builds a generic image suitable for computers with an ARM CPU (64 bit)

- **samus, eve, <your board name>** - builds an image specific to the chosen device (find your board name [here](#)); recommended for deploying to official hardware
- **betty** - (Googlers only) builds an ARC++-enabled image for running in a VM; recommended for Chromium developers.

To list all known boards in your checkout, run this command:

```
(outside)
$ cros query boards
```

You need to choose a board for your first build. Be aware that the generic images may not work well (or not at all) when run on official hardware. Don't worry too much about this choice, though – you can always build for another board later. If you want a list of known boards, you can look in `~/chromiumos/src/overlays`.

Each command in the build processes takes a `--board` parameter. To facilitate this, it can be helpful to keep the name of the board in a shell variable. This is not strictly necessary, but if you do this, you can simply copy and paste the commands below into your terminal program. Enter the following inside your shell:

```
(outside)
$ BOARD=<your pick of board>
```

This setting only holds while you stay in the chroot. If you leave and come back, you need to specify this setting again.

Build the packages for your board

To build all the packages for your board, run the following command:

```
(outside)
$ cros build-packages --board=${BOARD}
```

This step is the rough equivalent of `make all` in a standard Makefile system. This command handles incremental builds; you should run it whenever you change something and need to rebuild it (or after you run `repo sync`).

Normally, the `cros build-packages` command builds the stable version of a package (i.e. from committed git sources), unless you are working on a package (with `cros workon`). If you are working on a package, `cros build-packages` will build using your local sources. See below for information about `cros workon`.

SIDE NOTES:

- The first time you run `cros build-packages`, you should use `--autosetgov --autosetgov-sticky`. If your system uses a power-saving [CPU governor](#), these options will temporarily switch to `performance` for faster builds. `--autosetgov-sticky` makes `cros build-packages` remember to always use `--autosetgov`.
- (For Googlers) By default, with an internal manifest synced, `cros build-packages` will install either Chromium or Chrome, optimizing for *any* binpkg usage, falling back to building Chromium when no binpkg is available. The default behavior aims to be fast, but should not be relied on in cases where Chrome (or Chromium) is specifically required for your build. Add the `--chrome` flag to guarantee Chrome is installed, or the `--chromium` flag to guarantee Chromium. In all cases, binpkgs will be used whenever available, and the build will fall back to building from source if no matching binpkg is available.
- By default, packages other than Chrome will be compiled in debug mode; that is, with `NDEBUG` undefined and with debugging constructs like `DCHECK`, `DLOG`, and the red "Debug image" message present. If you supply `--no-withdebug`, then `NDEBUG` will be defined and the debugging constructs will be removed.

- The first time you run the `cros build-packages` command, it will take a long time (around 90 minutes on a four core machine), as it must build every package, and also download about 1.7GB of source packages and 1.3GB of binary packages. [See here](#) for more information about what the `cros build-packages` command actually does. In short, it tries to download existing binary packages to avoid building anything (and puts them in `/build/${BOARD}/packages` for subsequent builds). Failing that, it downloads source packages, puts them in `/var/lib/portage/distfiles-target`, and builds them.
- Should you want a clean build, pass `--cleanbuild`.
- Use `--no-usepkg` to build packages from source (required if you want to build a release builds spec). You may also want to set "USE=cros-debug" environment variable.
- If you don't want to have to pass `--board` to this and following commands, you can set a default board by creating the file `.default_board` in the `~/chromiumos/src/scripts` directory:

```
(outside)
$ echo ${BOARD} > ~/chromiumos/src/scripts/.default_board
```

Build a disk image for your board

Once the `cros build-packages` step is finished, you can build a ChromiumOS-base developer image by running the command below:

```
(outside)
$ cros build-image --board=${BOARD} --no-enable-rootfs-verification test
```

The arguments for `cros build-image` specify what type of build you want. A test image (in the example above) has additional test-specific packages and also accepts incoming SSH connections. It is more convenient to use test images, but developers could also build developer images. A developer image provides a ChromiumOS-based image with additional developer packages. To build it use `dev` instead of `test`. If building a test image, the password will be `test0000`. The `--no-enable-rootfs-verification` turns off verified boot allowing you to freely modify the root file system. The system is less secure using this flag, however, for rapid development you may want to set this flag. If you would like a more secure, locked-down version of ChromiumOS, then simply remove the `--no-enable-rootfs-verification` flag. Finally if you want just the pristine ChromiumOS-based image (closest to ChromeOS but not quite the same), pass in `base` rather than `test` or `dev`. Use `cros build-image --help` for more information.

The image produced by `cros build-image` will be located in `~/chromiumos/src/build/images/${BOARD}/versionNum/` (where `versionNum` will actually be a version number). The most recent image produced for a given board will be symlinked to `~/chromiumos/src/build/images/${BOARD}/latest`.

At the end of `cros build-image`'s output, it will print a command you can run to start the image in a `cros vm` virtual machine. It will print something like:

```
To run the image in a virtual machine, use:
cros vm --start --image-path=../build/images/amd64-generic/R111-15301.0.0-d2023_01_04_103804-a
1/chromiumos_test_image.bin --board=amd64-generic
```

Remember this command for future use; see [Running an image in a virtual machine](#).

IMPORTANT NOTE: It's up to you to delete old builds that you don't need. Every time you run `cros build-image`, the command creates files that take up to **8GB of space (!)**.

Installing ChromiumOS on your Device

Put your image on a USB disk

The easiest way to get your image running on your target computer is to put the image on a USB flash disk (sometimes called a USB key), and boot the target computer from the flash disk.

The first step is to disable auto-mounting of USB devices on your build computer as it may corrupt the disk image while it's being written. On systems that use GNOME or Cinnamon, run the following:

```
(outside)
$ gsettings set org.gnome.desktop.media-handling automount false
$ gsettings set org.gnome.desktop.media-handling automount-open false
$ gsettings set org.cinnamon.desktop.media-handling automount false
$ gsettings set org.cinnamon.desktop.media-handling automount-open false
```

Next, insert a USB flash disk (8GB or bigger) into your build computer. **This disk will be completely erased, so make sure it doesn't have anything important on it.** Wait ~10 seconds for the USB disk to register, then type the following command:

```
(outside)
$ cros flash usb:// ${BOARD}/latest
```

For more details on using this tool, see the [CrOS Flash page](#).

When the `cros flash` command finishes, you can simply unplug your USB key and it's ready to boot from.

IMPORTANT NOTE: To emphasize again, `cros flash` completely replaces the contents of your USB disk. Make sure there is nothing important on your USB disk before you run this command.

SIDE NOTES:

- If you want to create a test image (used for integration testing), see the [Running Tests](#) section.

Enter Developer Mode

See the [Developer Mode](#) documentation.

Boot from your USB disk

After enabling [Developer Mode](#), you should set your system to boot from USB.

Let the device boot, login and open a shell (or switch to terminal 2 via Ctrl+Alt+F2).

Run the following command:

```
(device)
$ sudo crossystem
```

You should see `dev_boot_usb` equal to 0. Set it to 1 to enable USB boot:

```
(device)
$ sudo crossystem dev_boot_usb=1
$ sudo crossystem dev_boot_signed_only=0
```

Now reboot. On the white screen (indicating [Developer Mode](#) is enabled), plug-in the USB disk and press [Ctrl+U](#) ([Debug Button Shortcuts](#)).

Getting to a command prompt on ChromiumOS

You have the ability to login as the chronos user:

1. After your computer has booted to the ChromiumOS login screen, press `[ Ctrl ] [ Alt ] [ F2 ]` to get a text-based login prompt. (On a Chromebook keyboard, `[ F2 ]` will appear as `[ ⌫ ]` or `[ → ]`.)
2. Log in with the chronos user and enter the password (usually `test0000`).

Because you built an image with developer tools, you also have an alternate way to get a terminal prompt. The alternate shell is a little nicer (in the very least, it keeps your screen from dimming on you), even if it is a little harder to get to. To use this alternate shell:

1. Go through the standard ChromiumOS login screen (you'll need to setup a network, etc.) and get to the web browser. It's OK to login as guest.
2. Press `[ Ctrl ] [ Alt ] [ T ]` to get the [crosh](#) shell.
3. Use the `shell` command to get the shell prompt. NOTE: you **don't** need to enter the `chronos` password here, though you will still need the password if you want to use the `sudo` command.

Installing your ChromiumOS image to your hard disk

Once you've booted from your USB key and gotten to the command prompt, you can install your ChromiumOS image to the hard disk on your computer with this command:

```
(device)
$ /usr/sbin/chromeos-install
```

IMPORTANT NOTE: Installing ChromiumOS onto your hard disk will **WIPE YOUR HARD DISK CLEAN**.

Running an image in a virtual machine

Many times it is easier to simply run ChromiumOS in a virtual machine like QEMU. You can use the [cros_vm](#) command to start a VM with the previously built image.

When you start the VM, `cros_vm` will print out information about how to connect to the running image via SSH and VNC.

For VNC it will normally say `VNC server running on ::1:5900` which means it's serving on localhost on the default VNC port (5900). You can connect to `localhost` with a VNC viewer to connect.

A good VNC client for Linux is the package `tigervnc-viewer` (available on at least Debian); its command line program is `vncviewer`. Note that before you use it, you should click **Options** → **Misc** → **Show dot when no cursor**. To download `tigervnc-viewer`, run:

```
(outside)
$ sudo apt-get install tigervnc-viewer
```

To connect to `localhost` using `vncviewer`, run:

```
(outside)
```

```
$ vncviewer localhost:5900 &
```

Other options include the VNC feature in [ChromiumIDE](#)'s device management, and [novnc](#), which makes the device accessible via a web browser.

***note **SIDE NOTES:**

- The output of the `cros build-image` command will also contain a full command to start a VM with that image.
- Only KVM/QEMU VMs are actively supported at the moment.
 - Non-KVM VMs often "work", but are unbearably slow.
- If you're interested in creating a test image (used for integration testing), see the [Running Tests](#) section.

Making changes to packages whose source code is checked into ChromiumOS git repositories

Now that you can build and run ChromiumOS, you're ready to start making changes to the code. For further hands on with making changes, you can check out the [build code lab](#).

NOTE: If you skipped to this section without building your own system image, you may run into hard-to-fix dependency problems if you build your own versions of system packages and try to deploy them to a system image that was built by a builder. If you run into trouble, try going through the full [Building ChromiumOS](#) process first and installing your own system image.

Keep the tree green

Before you start, take a moment to understand Chromium's source management strategy of "keeping the tree green". For the ChromiumOS project, **keeping the tree green** means:

1. Any new commits should not destabilize the build:
 - Images built from the tree should always have basic functionality working.
 - There may be minor functionality not working, and it may be the case, for example, that you will need to use Terminal to fix or work around some of the problems.
2. If you must introduce unstable changes to the tree (which should happen infrequently), you should use parameterization to hide new, unstable functionality behind a flag that's turned off by default. The ChromiumOS team leaders may need to develop mechanisms for parameterizing different parts of the system (such as the init script).
3. Internal "dev channel" releases will be produced directly from the tree, with a quick branch to check-point the release version. Any fixes required for a release will be pulled from the tree, avoiding merges back to tree.

This strategy has many benefits, including avoiding separate build trains for parallel development (and the cost of supporting such development), as well as avoiding large, costly merges from forked branches.

SIDE NOTE: "Keep the tree green" means something a bit different for ChromiumOS than for Chromium, which is much further along in its life cycle.

The steps in this section describe how to make changes to a ChromiumOS package whose source is checked into the ChromiumOS source control system. Specifically, this is a package where:

- The ****ebuild**** for the package lives in the `src/third_party/chromiumos-overlay` or `src/overlays/overlay-{BOARD}` directories.
- There is an ebuild for the package that ends with `9999.ebuild`.

- The ebuild inherits from the `cros-workon` class.
- The ebuild has a `KEYWORDS` in the ebuild containing this architecture name (like "x86").

You can see a list of all such packages by running the following command:

```
(outside)
$ cros workon --board=${BOARD} --all list
```

Run cros workon start

The first thing you need to do is to mark the package as active. Use the command below, replacing `${PACKAGE_NAME}` with your package name (e.g., `chromeos-wm`):

```
(outside)
$ cros workon --board=${BOARD} start ${PACKAGE_NAME}
```

This command:

- Indicates that you'd like to build the `9999` version of the `ebuild` instead of the stable, committed version.
- Indicates that you'd like to build from source every time.
- If you specified that you wanted the `minilayout` when you did your `repo init`, this command adds a clause to your `.repo/local_manifest.xml` to tell `repo` to sync down the source code for this package next time you do a `repo sync`.

Run repo sync

After running `cros workon`, sync down the sources. This is critical if you're using the `minilayout`, but is probably a good idea in any case to make sure that you're working with the latest code (it'll help avoid merge conflicts later). Run the command below anywhere under your `~/chromiumos` directory:

```
(outside)
$ repo sync
```

*** note **Note:** Make sure your `umask` is [set to a supported value](#) (e.g. 022); otherwise, you may end up with bad file permissions in your source tree.

Find out which ebuilds map to which directories

The `cros workon` tool can help you find out what ebuilds map to each directory. You can view a full list of ebuilds and directories using the following command:

```
(outside)
$ cros workon --board=${BOARD} --all info
```

If you want to find out which ebuilds use source code from a specific directory, you can use `grep` to find them. For example:

```
(outside)
$ cros workon --board=${BOARD} --all info | grep platform/ec
```

This returns the following output:

```
chromeos-base/ec-utils chromiumos/platform/ec src/platform/ec
```

This tells you the following information:

1. The name of the ebuild is `chromeos-base/ec-utils`
2. The path to the git repository on the server is `chromiumos/platform/ec`
3. The path to the source code on your system is `src/platform/ec`

You can similarly find what source code is associated with a given ebuild by grepping for the ebuild name in the list.

To find out where the ebuild lives:

```
(outside)
$ cros_sdk equery-${BOARD} which ${PACKAGE_NAME}
```

As an example, for `PACKAGE_NAME=ec-utils`, the above command might display:

```
/home/.../chromiumos/src/third_party/chromiumos-overlay/chromeos-base/ec-utils/ec-utils-9999.ebuild
```

SIDE NOTE: If you run the same command **without** running `cros workon` first, you can see the difference:

```
/home/.../chromiumos/src/third_party/chromiumos-overlay/chromeos-base/ec-utils/ec-utils-0.0.1-r134.ebuild
```

Create a branch for your changes

Since ChromiumOS uses `repo/git`, you should always create a local branch whenever you make changes.

First, find the source directory for the project you just used `cros workon` on. This isn't directly related to the project name you used with `cros workon`. (TODO: This isn't very helpful - someone with more experience, actually tell us how to find it reliably? --Meredydd)

cd into that directory, in particular the "files/" directory in which the actual source resides. In the command below, replace `${BRANCH_NAME}` with a name that is meaningful to you and that describes your changes (nobody else will see this name):

```
(outside)
$ repo start ${BRANCH_NAME} .
```

The branch that this creates will be based on the remote branch (TODO: which one? --Meredydd). If you've made any other local changes, they will not be present in this branch.

Make your changes

You should be able to make your changes to the source code now. To incrementally compile your changes, use either `cros_workon_make` or `emerge-${BOARD}`. To use `cros_workon_make`, run

```
(outside)
$ cros_sdk cros_workon_make --board=${BOARD} ${PACKAGE_NAME}
```

This will build your package inside your source directory. Change a single file, and it will rebuild only that file and re-link. If your package contains test binaries, using

```
(outside)
$ cros_sdk cros_workon_make --board=${BOARD} ${PACKAGE_NAME} --test
```

will build and run those binaries as well. In case your tests have out-of-package dependencies, you'll first need to run

```
(outside)
$ cros_sdk USE=test emerge-${BOARD} ${PACKAGE_NAME}
```

to pull them in.

Call `cros_workon_make --help` to see other options that are supported.

You probably want to get your changes onto your device now. You need to install the changes you made by using

```
(outside)
$ cros_sdk cros_workon_make --board=${BOARD} ${PACKAGE_NAME} --install
```

You can then rebuild an image with `cros build-image` and reimage your device.

Alternatively, you can build your package using `emerge-${BOARD}` and quickly install it to the device by using [cros deploy](#).

For example, if you want to build `ec-utils` to test on your device, use

```
(outside)
$ cros_sdk emerge-${BOARD} ec-utils
```

To install the package to the device, use

```
(outside)
$ cros deploy ${IP} ec-utils
```

Set your editor

Many of the commands below (in particular `git`) open up an editor. You probably want to run one of the three commands below depending on your favorite editor.

If you're not a *nix expert, `nano` is a reasonable editor:

```
$ export EDITOR='nano'
```

If you love `vi`:

```
$ export EDITOR='vi'
```

If you love `emacs` (and don't want an X window to open up every time you do something):

```
$ export EDITOR='emacs -nw'
```

You should probably add one of those lines to your `.bashrc` (or similar file) too.

Submit changes locally

When your changes look good, commit them to your local branch using `git`. Full documentation of how to use git is beyond the scope of this guide, but you might be able to commit your changes by running something like the command below from the project directory:

```
(outside)
$ git commit -a
```

The `git commit` command brings up a text editor. You should describe your changes, save, and exit the editor. Note that the description you provide is only for your own use. When you upload your changes for code review, the `repo upload` command grabs all of your previous descriptions, and gives you a chance to edit them.

Upload your changes and get a code review

Check out our [Gerrit Workflow](#) guide for details on our review process.

Clean up after you're done with your changes

After you're done with your changes, you're ready to clean up. The most important thing to do is to tell `cros workon` that you're done by running the following command:

```
(outside)
$ cros workon --board=${BOARD} stop ${PACKAGE_NAME}
```

This command tells `cros workon` to stop forcing the `-9999.ebuild` and to stop forcing a build from source every time.

If you're using the `minilayout`, doing a `cros workon stop` **will not** remove your source code. The code will continue to stay on your hard disk and get synced down.

Making changes to non-cros-workon-able packages

If you want to make changes to something other than packages which source is checked into the ChromiumOS source control system, you can follow the instructions in the previous section, but skip the `cros workon` step. Note specifically that you still need to run `repo start` to [Create a branch for your changes](#).

The types of changes that fall into this category include:

- changes to build scripts (pretty much anything in `src/scripts`)
- changes to `ebuild` files themselves (like the ones in `src/third_party/chromiumos-overlay`)
 - To change these files, you need to "manually uprev" the package. For example, if we're making a modification to for the Pixel overlay (link), then

- `cd src/overlays/overlay-link/chromeos-base/chromeos-bsp-link`
- `mv chromeos-bsp-link-0.0.2-r29.ebuild chromeos-bsp-link-0.0.2-r30.ebuild`
- `chromeos-bsp-link-0.0.2-r29.ebuild` is a symlink that points to `chromeos-bsp-link-0.0.2.ebuild`. To uprev the package, simply increment the revision (r29) number.
- Note: Upreving should not be done when there is an ebuild for the package that ends with `9999.ebuild`. Changes to the ebuild should usually be done in the `9999.ebuild` file.
- adding small patches to existing packages whose source code is NOT checked into ChromiumOS git repositories
- changes to `eclass` files (like the ones in `src/third_party/chromiumos-overlay/eclass`)
- changes to the buildbot infrastructure (in `crostools`)
- changes to ChromiumOS project documentation (in `docs`)
- TODO: anything else?

Adding small patches to existing packages

When you need to add small patches to existing packages whose source code is not checked into a ChromiumOS git repository (e.g. it comes from portage, and is not a `crosworkon`-able package), you need to do the following:

First, find the package ebuild file under `third_party/chromiumos-overlay`.

Then, [create a patch file](#) from the exact version of the package that is used by the current ebuild. If other patches are already in the ebuild, you'll want to add your patch LAST, and build the patch off of the source that has already had the existing patches applied (either do it by hand, or set `FEATURES=noclean` and build your patch off of the temp source). Note that patch order is significant, since the ebuild expects each patch line number to be accurate after the previous patch is applied.

Place your patch in the "files" subdir of the directory that contains the ebuild file (e.g. `third_party/chromiumos-overlay/dev-libs/mypackage/files/mypackage-1.0.0-my-little-patch.patch`).

Then, in the `prepare()` section of the ebuild (create one if it doesn't exist), add an `epatch` line:

```
$ epatch "${FILESDIR}/${P}-my-little-patch.patch
```

Lastly, you'll need to bump the revision number in the name of the ebuild file (or symlink) so the build system picks up the change. The current wisdom is that the ebuild file should be symlinked instead of being renamed. For example, if the original ebuild file is `"mypackage-1.0.0.ebuild"`, you should create a `"mypackage-1.0.0-r1.ebuild"` symbolic link that points at the original ebuild file. If that symlink already exists, create the next higher "rN" symlink.

Making changes to the way that the chroot is constructed

TODO: This section is currently a placeholder, waiting for someone to fill it in. However, a few notes:

- Many of the commands that take a `--board=${BOARD}` parameter also take a `--host` parameter, which makes the commands affect the host (i.e. the chroot) rather than the board.
 - Most notably, `crosworkon --host` says that you want to build a package used in the chroot from source.

Building an individual package

TODO: Document this better, and add the new `crosworkon_make`.

SIDE NOTE: To build an individual portage package, for a particular board, use `emerge-${BOARD}`.

For example, if you want to build dash to test on your device:

```
(outside)
$ cros_sdk emerge-${BOARD} dash
```

To install the package to the device, see [cros deploy](#).

SIDE NOTE:

- Typically, when building a package with `emerge-${BOARD}`, the dependencies have already been built. However, in some situations dependencies will need to be built as well. When that happens, `-jN` can be passed to `emerge-${BOARD}` to build different packages in parallel.

Making changes to the Chromium web browser on ChromiumOS

If you want to make modifications to the Chromium web browser and quickly deploy your changes to an already-built ChromiumOS image, see the [Simple Chrome Workflow](#). Or if you have a pending Chromium change that you'd like tested remotely on CrOS's CQ before submitting it, see the [chromeos-uprev-tester](#) trybot.

Local Debugging

There are two options for debugging: `cros debug` and `gdb-${BOARD}`. Both tools support local debugging within your board sysroot, and both use the proper board-specific libraries, thus allowing you to run your target compiled binaries locally.

`cros debug` is intended to be a modern replacement for `gdb-${BOARD}`, defaulting to the LLVM debugger (LLDB) instead of GDB. It handles both remote and local debugging, and automatically points the debugger to your debug symbol files and source code files. Although `gdb-${BOARD}` is still available, we expect you will have a better experience with the much newer `cros debug`.

Debugging both x86 and non-x86 binaries on your workstation

Using the `cros debug` tool

`cros debug`, when launched without a `--device` argument, will default to local debugging, using QEMU if the target binary's architecture does not match that of your development machine. In this mode, a `--board` must be specified. This will ensure the debugger is using the correct dependencies and libraries by creating a sysroot rooted in your board's build directory.

In local debugging mode, `cros debug` uses the board-specific `lldb` binary, as the debugger itself runs in the sysroot within the board's build directory. Therefore, you should ensure you have built the local `lldb` binary for your board, like so:

```
(outside)
USE="local-lldb" cros build-packages --board=${BOARD} dev-util/lldb-server
```

If you want command history and line editing support in local debugging, add the `libedit` USE flag:

```
(outside)
USE="local-lldb libedit" cros build-packages --board=${BOARD} dev-util/lldb-server
```

An example invocation, which locally starts `$VOLTEER_BUILD/usr/sbin/spaced` within `lldb` inside the `volteer` sysroot:

```
(outside)
$ cros debug --board=volteer --exe=/usr/sbin/spaced
```

Note that QEMU can often be unreliable for debugging, or fail to work at all. Note that you may need to resort to remote debugging on actual hardware (e.g. using `crosfleet`) when debugging on non-x86 architectures.

Using `gdb-${BOARD}`

If you build your projects incrementally, write unit tests and use them to drive your development, you may want to debug your code without shipping it over to a running device or VM.

`gdb-${BOARD}` sets up `gdb` in your board sysroot and ensures that `gdb` is using the proper libraries, debug files, etc. for debugging, allowing you to run your target-compiled binaries.

It should already be installed in your chroot. If you do not have the script, update your repository to get the latest changes, then re-build your packages:

```
(outside)
$ repo sync

(outside)
$ cros build-packages --board=...
```

This should install `gdb-${BOARD}` in the `/usr/local/bin` directory inside the chroot. These board-specific `gdb` wrapper scripts correctly handle *both local and remote debugging* (see next section for more information on remote debugging). When used for local debugging, these scripts will run inside a special chroot-inside-your-chroot, rooted in the board's sysroot. For example if you are using `gdb-lumpy`, it will run inside a chroot based entirely in your `/build/lumpy` sysroot. *The libraries that it will load and use are the libraries in the sysroot*, i.e. the target libraries for the board; the `gdb` binary it will use is the `gdb` binary in that tree. While debugging with `gdb-lumpy` (for local debugging), you will not be able to see/access any files outside of the `/build/lumpy` tree. While for the most part this is very good, as it ensures the correct debugging environment, it does mean that if you want to use this script to debug a `lumpy` binary, such as a unit test, that you built outside of the `/build/lumpy` tree, you will need to copy the binary to the `/build/lumpy` tree first. Also, if you want the debugger to be able to see the source files when debugging, you will need to make sure they exist inside the `/build/lumpy` tree as well (see example below).

IMPORTANT NOTE 1: Local and remote debugging functionality are combined in this single script. Some of the options shown below only work for remote debugging.

IMPORTANT NOTE 2: When doing local debugging of x86 binaries, they will try to execute on your desktop machine (using the appropriate libraries and `gdb` binaries). It is possible that for some x86 boards, the binaries may use instructions not understood by your hardware (particularly some vector instructions), in which case you will need to do remote debugging with the actual hardware instead.

IMPORTANT NOTE 3: You can use this script with **some** debugging functionality for local debugging of non-x86 binaries. The script loads QEMU and runs the non-x86 binaries in QEMU. However QEMU has some unfortunate limitations. For example you can "set" breakpoints in the binary (to see what addresses correspond to locations in the source), examine the source or assembly code, and execute the program. But QEMU does not actually hit the breakpoints, so you cannot suspend execution in the middle when running under QEMU. For full debugging functionality with non-x86 binaries, you must debug them remotely running on the correct hardware (see next section on remote debugging). You can see this in the example below, where `gdb-daisy` does not actually stop at the breakpoint it appears to set, although it does correctly execute the program.

```
(outside)
$ cros_sdk gdb-daisy -h

usage: cros_gdb [-h]
               [--log-level {fatal,critical,error,warning,notice,info,debug}]
               [--log-format LOG_FORMAT] [--debug] [--nocolor]
               [--board BOARD] [-g GDB_ARGS] [--remote REMOTE] [--pid PID]
               [--remote-pid PID] [--no-ping] [--attach ATTACH_NAME] [--cgdb]
               [binary-to-be-debugged] [args-for-binary-being-debugged]
```

Wrapper **for** running gdb.

This handles the fun details like running against the right sysroot, via QEMU, **bind** mounts, etc...

positional arguments:

inf_args Arguments **for** gdb to pass to the program being debugged. These are positional and must come at the end of the **command** line. This will not work **if** attaching to an already running program.

...

```
$ cros_sdk gdb-daisy /bin/grep shebang /bin/ls
15:51:06: INFO: run: file /build/daisy/bin/grep
Reading symbols from /bin/grep...Reading symbols from /usr/lib/debug/bin/grep.debug...done.
(daisy-gdb) b main
Breakpoint 1 at 0x2814: file grep.c, line 2111.
(daisy-gdb) disass main
Dump of assembler code for function main:
   0x00002814 <+0>: ldr.w r2, [pc, #3408] ; 0x3568 <main+3412>
   0x00002818 <+4>: str.w r4, [sp, #-36]!
   0x0000281c <+8>: movs r4, #0
   0x0000281e <+10>: strd r5, r6, [sp, #4]
   0x00002822 <+14>: ldr.w r3, [pc, #3400] ; 0x356c <main+3416>
   0x00002826 <+18>: movs r5, #2
   0x00002828 <+20>: strd r7, r8, [sp, #12]
...
(daisy-gdb) run
Starting program: /bin/grep shebang /bin/ls
qemu: Unsupported syscall: 26
#!/usr/bin/coreutils --coreutils-prog-shebang=ls
qemu: Unsupported syscall: 26
During startup program exited normally.
(daisy-gdb) quit
```

Note in the example above that, like "regular" gdb when given **--args**, you can pass the arguments for the program being debugged to the gdb wrapper script just by adding them to the command line after the name of the program being debugged (except that **--args** isn't needed).

The commands below show how to copy your incrementally-compiled unit test binary and source file(s) to the appropriate sysroot and then start gdb with that binary (using the correct libraries, etc).

```
(outside)
$ cd ~/chromiumos/chroot/build/lumpy/tmp/portage
```

```
$ mkdir shill-test
$ cd shill-test
$ cp <path-to-binary>/shill_unittest .
$ cp <path-to-src>/shill_unittest.cc .
$ cros_sdk gdb-lumpy
(gdb-lumpy) directory /tmp/portage/shill-test # Tell gdb to add /tmp/portage/shill-test to the paths it searches for source files
(gdb-lumpy) file ./shill_unittest
```

If gdb is still looking for the source file in the wrong directory path, you can use `set substitute-path <from> <to>` inside gdb to help it find the right path (inside your sysroot) for searching for source files.

Printing stack traces at runtime

See [Stack Traces](#) for how to print stack traces at runtime.

Remote Debugging

Setting up remote debugging by hand

If you want to manually run through all the steps necessary to set up your system for remote debugging and start the debugger, see [Remote Debugging in ChromiumOS](#).

Automated remote debugging using `cros debug`

`cros debug` can automate many of the required steps for remote debugging using `lldb` with a single command, as long as you have an SSH connection to the remote device. `cros debug` aims to be a replacement for `gdb-${board}` with support for LLDB.

`cros debug`---in remote debugging mode---works by doing the following:

- establishing the required SSH tunnel(s) and starting the debug server,
- running a local instance of the debugger client,
- and automatically connecting to the remote server.

Note that the tool runs the local debug client inside the chroot so that locally built debug symbols and source files can be found by the debugger client without requiring symbols to be present on the remote device.

- By leveraging LLDB's "platform" mode, `cros debug` can enable a quick development and remote debugging cycle without the need to `cros deploy` your updated binary each time you make changes. LLDB will seamlessly copy the debug target binary to the remote device from the local development machine. LLDB checks the hash of the binary so that it is only copied if the binary has changed. If the binary on the remote device is up to date (as it would be right after a `cros deploy`), copying is not done. However, this workflow has some limitations:
 - If your debug target program has additional dependency files, these must be copied to the remote device each time. This however can be done from inside `lldb`. More information about remote debugging features in `lldb` can be found in the [LLDB documentation](#).
 - You must compile the binary for the *target* architecture, which may differ from the local machine's architecture.

Some common invocation examples:

Debug a particular executable:

```
(outside)
cros debug --device $DUT --exe /path/to/exe
```

Attach the debugger to the currently running process with PID 1234 (note that `-d` is shorthand for `--device`):

```
(outside)
cros debug -d $DUT --pid 1234
```

List all processes of a particular executable, without launching a debugger:

```
(outside)
cros debug -d $DUT --exe /path/to/exe --list
```

Use a binary that is only present on the remote device (note that `--debugger=lldb` is the default):

```
(outside)
cros debug -d $DUT --exe /path/to/remote/exe/ --use-remote-exe --debugger lldb
```

- If there are any already running processes of this executable, then they will be listed, and you may choose to attach to one of them or launch a new process.

Run the debugger with additional command line arguments with the `--debug-arg` or `-g` flag. These arguments are properly shell quoted and appended, in the order they are provided, to the local debugger launch command, after all other arguments:

```
(outside)
cros debug -d $DUT --exe /path/to/exe -g---source-quietly -g"--arch aarch64"
```

Launch the local debugger with a coredump file (note the path is again rooted in the board's build directory):

```
(outside)
cros debug --board $BOARD --exe /path/to/exe --corefile /path/to/core/dump
```

Setup notes:

Debug symbols will be autodetected as long as they have been built (this is the default). Additional debug symbol path search locations can be added to LLDB like so (note that these paths are rooted in the board's sysroot):

```
(lldb) settings append target.debug-file-search-paths /path/to/debug/files
```

Source code should also be found as long as it is still present in your `build` directory. This requires that the package was emerged with `FEATURES="noclean"`, which you can achieve like so, substituting your `${BOARD}` and desired `${package}`:

```
cros_sdk FEATURES="noclean" -- emerge-${BOARD} ${package}
```

`cros deploy` this package and source code files should now be found by the debugger when you next run `cros debug`.

`cros debug` command line options summary

- `-l, --list`: Lists all processes of the given `--exe` executable currently running on the remote device and exit, without launching a debugger.
- `--use-remote-exe`: When using `--debugger=lldb`, interpret the path provided to `--exe` as a remote-only path, and therefore use the `-r` flag when running LLDB's `target create`. This bypasses the automatic moving of your binary from the local device to the remote device provided by LLDB's "platform" connection.
- `--debugger-path`: Provide a path, rooted in the board's sysroot, to a specific local debugger binary. Otherwise, this will use the same name as is provided to `--debugger`. For example, `--debugger-path=/usr/local/bin/lldb`.
- `--debugger {lldb,gdb}`: Specify whether to use LLDB or GDB. While there are extension points for future GDB integration, GDB is not currently supported.
- `--gdbserver-port`: Specify a port number to use for the gdbserver connection. This is used for both LLDB and GDB.
- `--platform-port-local`: Specify the local port number for the LLDB platform connection. This is distinct from the `gdbserver` port required by LLDB. This local port number will be forwarded to the remote platform port number on the target device through an SSH tunnel.
- `--platform-port-remote`: Specify the remote port number for the LLDB platform connection.

Example: debugging a binary locally with `cros debug`

Debugging locally can be particularly helpful when doing iterative development, as you do not need to rely on deploying each change to a remote device. You can debug locally like the following example using the `volteer` board. Output-only text is commented out to highlight user input:

```
(outside)
$ cros debug --board=volteer --exe=/usr/bin/dig

# (lldb) platform settings -w /build/volteer
# (lldb) platform select --sysroot /build/volteer host
# Platform: host
# Triple: x86_64-*-linux-gnu
# OS Version: 6.7.12 (6.7.12-1rodete1-amd64)
# Hostname: 127.0.0.1
# Sysroot: /build/volteer
# WorkingDir: /build/volteer
# Kernel: #1 SMP PREEMPT_DYNAMIC Debian 6.7.12-1rodete1 (2024-06-12)
# Kernel: Linux
# Release: 6.7.12-1rodete1-amd64
# Version: #1 SMP PREEMPT_DYNAMIC Debian 6.7.12-1rodete1 (2024-06-12)
# (lldb) settings append target.debug-file-search-paths /build/volteer/usr/lib/debug
# (lldb) settings set prompt "(lldb-volteer) "
# (lldb-volteer) target create /usr/bin/dig
# Current executable set to '/usr/bin/dig' (x86_64).
# Could not load history file
(lldb-volteer) r
# Process 266 launched: '/usr/bin/dig' (x86_64)
#
# ; <<>> DiG 9.16.42 <<>>
# ;; global options: +cmd
# ;; Got answer:
# ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14005
# ;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1
#
# ;; OPT PSEUDOSECTION:
# ; EDNS: version: 0, flags:; udp: 1232
```

```
# ;; QUESTION SECTION:
# ;;                               IN    NS
#
# ;; ANSWER SECTION:
# .           87198 IN      NS      j.root-servers.net.
# .           87198 IN      NS      f.root-servers.net.
# .           87198 IN      NS      k.root-servers.net.
# .           87198 IN      NS      h.root-servers.net.
# .           87198 IN      NS      m.root-servers.net.
# .           87198 IN      NS      l.root-servers.net.
# .           87198 IN      NS      d.root-servers.net.
# .           87198 IN      NS      i.root-servers.net.
# .           87198 IN      NS      g.root-servers.net.
# .           87198 IN      NS      c.root-servers.net.
# .           87198 IN      NS      e.root-servers.net.
# .           87198 IN      NS      a.root-servers.net.
# .           87198 IN      NS      b.root-servers.net.
#
# ;; Query time: 95 msec
# ;; SERVER: 127.0.0.1#53(127.0.0.1)
# ;; WHEN: Thu Aug 29 22:01:40 UTC 2024
# ;; MSG SIZE rcvd: 431
#
# Process 266 exited with status = 0 (0x00000000)
(lldb-volteer)
```

This example merely runs the target program with the `r` command, but once the `(lldb-volteer)` prompt appears, you have full control of LLDB and can set breakpoints or issue any other LLDB commands. Consult the [LLDB documentation](#) for further information on how to use LLDB.

Example: debugging a running process on a remote device with `cross debug`

If you know there is an instance of an executable running on the remote device, but do not know the PID of that process, `cross debug` will list out the running instances of that executable when using the `--exe` option, and will give you the choice to attach to any one of them, or to start a new process.

If no processes of that executable are currently running, `lldb` will start one when you issue the `run` command.

Below is an example invocation. Once the first command is issued and the process is selected, the rest of the commands are handled automatically. Output-only text is commented out below to highlight the places where user input is required.

```
(outside)
$ cross debug -d $DUT --exe=/usr/sbin/spaced

# List running processes of /usr/sbin/spaced on device cri26-8:
# USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
# spaced    1885  0.0  0.3 24680 12844 ?        S    11:21   0:00 /usr/sbin/spaced
# Please select the process pid to debug (select [0] to start a new process):
#  [0]: Start a new process under LLDB
#  [1]: 1885
Please choose an option [0-1]: 1
# Warning: Permanently added 'cri26-8.cr.iad65.cr.dev' (ED25519) to the list of known hosts.
# (lldb) platform select --sysroot /build/volteer remote-linux
# Platform: remote-linux
```

```
# Connected: no
# Sysroot: /build/volteer
# (lldb) settings append target.debug-file-search-paths /build/volteer/usr/lib/debug
# (lldb) platform connect connect://localhost:58705
# Platform: remote-linux
# Triple: x86_64-unknown-linux-gnu
# OS Version: 5.4.282 (5.4.282-23086-g8c9d0527d37d)
# Hostname: localhost
# Connected: yes
# Sysroot: /build/volteer
# WorkingDir: /root
# Kernel: #1 SMP PREEMPT Thu, 29 Aug 2024 00:28:26 +0000
# (lldb) settings set prompt "(lldb-volteer) "
# (lldb-volteer) attach 1885
# Process 1885 stopped
# * thread #1, name = 'spaced', stop reason = signal SIGSTOP
# frame #0: 0x00007d7e16c08403 libc.so.6`epoll_wait + 19
# libc.so.6`epoll_wait:
# -> 0x7d7e16c08403 <+19>: cmpq  $-0x1000, %rax ; imm = 0xF000
# 0x7d7e16c08409 <+25>: ja 0x7d7e16c08460 ; <+112>
# 0x7d7e16c0840b <+27>: retq
# 0x7d7e16c0840c <+28>: nopl (%rax)
# Executable module set to "/home/abargher/.lldb/module_cache/remote-linux/.cache/460E200B-2A37-33D7/spaced".
# Architecture set to: x86_64-unknown-linux-gnu.
(lldb-volteer)
```

Tips For Googlers

Googlers may find the SSH setup instructions [here](#) helpful, especially for connecting to DUTs which require corp-ssh. [corp-ssh-helper-helper](#) creates the most seamless experience with `cros debug`.

Automated remote debugging using `gdb-${BOARD}` script (gdb-lumpy, gdb-daisy, gdb-parrot, etc)

`gdb-${BOARD}` is a script that automates many of the steps necessary for setting up remote debugging with gdb. It should already be installed in your chroot. If you do not have the script, update your repository to get the latest changes, then re-build your packages:

```
(outside)
$ repo sync

(outside)
$ cros build-packages --board=...
```

This should install `gdb_remote` in the `/usr/bin` directory inside the chroot. The `gdb-${BOARD}` script takes several options. The most important ones are mentioned below.

`--gdb_args (-g)` are arguments to be passed to gdb itself (rather than to the program gdb is debugging). If multiple arguments are passed, each argument requires a separate `-g` flag.

E.g `gdb-lumpy --remote=123.45.67.765 -g "-core=/tmp/core" -g "-directory=/tmp/source"`

`--remote` is the `ip_address` or name for your Chromebook, if you are doing remote debugging. If you omit this argument, the assumption is you are doing local debugging in the `sysroot` on your desktop (see section above). If you are debugging in the VM, then you need to specify either `:vm:` or `localhost:9222`.

`--pid` is the PID of a running process on the remote device to which you want `gdb/gdbserver` to attach.

`--attach` is the name of the running process on the remote device to which you want `gdb/gdbserver` to attach. If you want to attach to the Chrome browser itself, there are three special names you can use: `browser` will attach to the main browser process; `gpu-process` will attach to the GPU process; and `renderer` will attach to the renderer process if there is only one. If there is more than one renderer process `--attach=renderer` will return a list of the renderer PIDs and stop.

To have `gdb/gdbserver` start and attach to a new (not already running) binary, give the name of the binary, followed by any arguments for the binary, at the end of the command line:

```
(outside)
$ cros_sdk gdb-daisy --remote=123.45.67.809 /bin/grep "test" /tmp/myfile
```

When doing remote debugging you *must* use the `--pid` or the `--attach` option, or specify the name of a new binary to start. You cannot start a remote debugging session without having specified the program to debug in one of these three ways.

When you invoke `gdb-${BOARD} --remote=...`, it will connect to the notebook or VM (automatically setting up port-forwarding on the VM), make sure the port is entered into the iptables, and start up `gdbserver`, using the correct port and binary, either attaching to the binary (if a remote PID or name was specified) or starting up the binary. It will also start the appropriate version of `gdb` (for whichever type of board you are debugging) on your desktop and connect the `gdb` on your desktop to the `gdbserver` on the remote device.

Edit/Debug cycle

If you want to edit code and debug it on the DUT you can follow this procedure:

For `cros debug`:

```
(outside)
$ cros_sdk CFLAGS='-g' FEATURES='noclean' -- emerge-${BOARD} -v sys-apps/mosys
$ cros deploy --board=${BOARD} ${IP} sys-apps/mosys
$ cros debug --device ${IP} /usr/sbin/mosys
```

For `gdb-${BOARD}`:

```
(outside)
$ cros_sdk CFLAGS="-ggdb" FEATURES="noclean" emerge-${BOARD} -v sys-apps/mosys
$ cros deploy --board=${BOARD} ${IP} sys-apps/mosys
$ cros_sdk gdb-${BOARD} --cgdb --remote "${IP}" ¥
  -g "--eval-command=directory /build/${BOARD}/tmp/portage/sys-apps/mosys-9999/work/" ¥
  /usr/sbin/mosys -V
```

This will build your package with debug symbols (assuming your package respects `CFLAGS`). We need to use the `noclean` feature so that we have access to the original sourcecode that was used to build the package. Some packages will generate build artifacts and have different directory structures than the tar/git repo. This ensures all the paths line up correctly and the source code can be located. Ideally we would use the `installsources` feature, but we don't have support for the `debugedit` package (yet!). Portage by default will strip the symbols and install the debug symbols in `/usr`

`r/lib/debug/`. `cros debug` or `gdb-${BOARD}` will handle setting up the correct debug symbol path. `cros deploy` will then update the rootfs on the DUT. In the `gdb-${BOARD}` example, we pass the work directory into `gdb-${BOARD}` so that `cgdb` can display the sourcecode inline.

Quick primer on `cgdb`:

- `ESC`: moves to the source window.
- `i`: moves from source window to `gdb` window.

Examples of debugging using the `gdb-${BOARD}` script

Below are three examples of using the board-specific `gdb` wrapper scripts to start up debugging sessions. The first two examples show connecting to a remote Chromebook. The first one automatically finds the browser's running GPU process, attaches `gdbserver` to the running process, starts `gdb` on the desktop, and connects the local `gdb` to `gdbserver`. It also shows the user running the `bt` (backtrace) command after `gdb` comes up. The second example shows the user specifying the PID of a process on the Chromebook. Again the script attaches `gdbserver` to the process, starts `gdb` on the desktop, and connects the two. The third example shows the user connecting to the main browser process in ChromeOS running in a VM on the user's desktop. For debugging the VM, you can use either `--remote=vm`: or `--remote=localhost:9222` (`vm`: gets translated into `localhost:9222`).

Example 1:

```
(outside)
$ cros_sdk gdb-lumpy --remote=123.45.67.809 --attach=gpu-process

14:50:07: INFO: run: ping -c 1 -w 20 123.45.67.809
14:50:09: INFO: run: file /build/lumpy/opt/google/chrome/chrome
14:50:10: INFO: run: x86_64-cros-linux-gnu-gdb --quiet '--eval-command=set sysroot /build/lumpy' '--eval-command=set solib-absolute-prefix /build/lumpy' '--eval-command=set solib-search-path /build/lumpy' '--eval-command=set debug-file-directory /build/lumpy/usr/lib/debug' '--eval-command=set prompt (lumpy-gdb) ' '--eval-command=file /build/lumpy/opt/google/chrome/chrome' '--eval-command=target remote localhost:38080'
Reading symbols from /build/lumpy/opt/google/chrome/chrome...Reading symbols from/build/lumpy/usr/lib/debug/opt/google/chrome/chrome.debug...done.
(lumpy-gdb) bt
#0 0x00007f301fad56ad in poll () at ../sysdeps/unix/syscall-template.S:81
#1 0x00007f3020d5787c in g_main_context_poll (priority=2147483647, n_fds=3, fds=0xdce10719840, timeout=-1, context=0xdce1070ddc0) at gmain.c:3584
#2 g_main_context_iterate (context=context@entry=0xdce1070ddc0, block=block@entry=1, dispatch=dispatch@entry=1, self=<optimized out>) at gmain.c:3285
#3 0x00007f3020d5798c in g_main_context_iteration (context=0xdce1070ddc0, may_block=1) at gmain.c:3351
#4 0x00007f30226a4c1a in base::MessagePumpGlib::Run (this=0xdce10718580, delegate=<optimized out>) at ../../../../chromeos-cache/distfiles/target/chrome-src-internal/src/base/message_loop/message_pump_glib.cc:309
#5 0x00007f30226666ef in base::RunLoop::Run (this=this@entry=0x7fff72271af0) at ../../../../chromeos-cache/distfiles/target/chrome-src-internal/src/base/run_loop.cc:55
#6 0x00007f302264e165 in base::MessageLoop::Run (this=this@entry=0x7fff72271db0) at ../../../../chromeos-cache/distfiles/target/chrome-src-internal/src/base/message_loop/message_loop.cc:307
#7 0x00007f30266bc847 in content::GpuMain (parameters=...) at ../../../../chromeos-cache/distfiles/target/chrome-src-internal/src/content/gpu/gpu_main.cc:365
#8 0x00007f30225cedee in content::RunNamedProcessTypeMain (process_type=..., main_function_params=..., delegate=0x7fff72272380) at ../../../../chromeos-cache/distfiles/target/chrome-src-internal/src/content/app/content_main_runner.cc:385
#9 0x00007f30225cef3a in content::ContentMainRunnerImpl::Run (this=0xdce106fef50) at ../../../../c
```

```
hromeos-cache/distfiles/target/chrome-src-internal/src/content/app/content_main_runner.cc:763
#10 0x00007f30225cd551 in content::ContentMain (params=...) at ../../../../chromeos-cache/distfiles/
target/chrome-src-internal/src/content/app/content_main.cc:19
#11 0x00007f3021fef02a in ChromeMain (argc=21, argv=0x7fff722724b8) at ../../../../chromeos-cach
e/distfiles/target/chrome-src-internal/src/chrome/app/chrome_main.cc:66
#12 0x00007f301fa0bf40 in __libc_start_main (main=0x7f3021fee760 <main(int, char const**)>, argc=2
1, argv=0x7fff722724b8, init=<optimized out>, fini=<optimized out>, rtdl_fini=<optimized out>, stack_e
nd=0x7fff722724a8) at libc-start.c:292
#13 0x00007f3021feee95 in _start ()
(lumpy-gdb)
```

Example 2:

```
(outside)
$ cros_sdk gdb-daisy --pid=626 --remote=123.45.98.765
14:50:07: INFO: run: ping -c 1 -w 20 123.45.98.765
14:50:09: INFO: run: file /build/daisy/usr/sbin/cryptohomed
14:50:10: INFO: run: armv7a-cros-linux-gnueabi-gdb --quiet '--eval-command=set sysroot /build/daisy'
'--eval-command=set solib-absolute-prefix /build/daisy' '--eval-command=set solib-search-path /buil
d/daisy' '--eval-command=set debug-file-directory /build/daisy/usr/lib/debug' '--eval-command=set pr
ompt (daisy-gdb) '--eval-command=file /build/daisy/usr/sbin/cryptohomed' '--eval-command=target r
emote localhost:38080'
Reading symbols from /build/daisy/usr/sbin/cryptohomed...Reading symbols from /build/daisy/usr/lib/
debug/usr/bin/cryptohomed.debug...done.
(daisy-gdb)
```

Example 3:

```
(outside)
$ cros_sdk gdb-lumpy --remote=:vm: --attach=browser
15:18:28: INFO: run: ping -c 1 -w 20 localhost
15:18:31: INFO: run: file /build/lumpy/opt/google/chrome/chrome
15:18:33: INFO: run: x86_64-cros-linux-gnu-gdb --quiet '--eval-command=setsysroot /build/lumpy' '--ev
al-command=set solib-absolute-prefix /build/lumpy' '--eval-command=set solib-search-path /build/lu
mpy' '--eval-command=set debug-file-directory /build/lumpy/usr/lib/debug' '--eval-command=set pro
mpt (lumpy-gdb) '--eval-command=file /build/lumpy/opt/google/chrome/chrome' '--eval-command=t
arget remote localhost:48062'
Reading symbols from /build/lumpy/opt/google/chrome/chrome...Reading symbols from /build/lumpy/
usr/lib/debug/opt/google/chrome/chrome.debug...done.
done.
Remote debugging using localhost:48062
...
(lumpy-gdb)
```

If you find problems with the board-specific gdb scripts, please file a bug (crbug.com/new) and add 'build-toolchain' as one of the labels in the bug.

Troubleshooting

I lost my developer tools on the stateful partition, can I get them back?

This happens sometimes because the security system likes to wipe out the stateful partition and a lot of developer tools are in /usr/local/bin. You may be able restore some of them using the `dev-install` tool. See the [dev-install docs](#)

for more information about its "Base Image" workflow.

Disabling Enterprise Enrollment

Some devices may be configured with a policy that only allows logging in with enterprise credentials, which will prevent you from logging in with a non-enterprise Google account (e.g., foo@gmail.com). To disable the enterprise enrollment setting:

- Enable [Developer Mode](#).
- Disable the enterprise enrollment check:

```
(device)
$ vpd -i RW_VPD -s check_enrollment=0
$ crossystem clear_tpm_owner_request=1
$ reboot
```

***note **NOTE:** The enterprise policy can also prevent transitioning to [Developer Mode](#), in which case you won't be able to perform the above commands.

Running Tests

ChromiumOS integration (or "functional") tests are written using the [Tast](#) or [Autotest](#) frameworks.

Set up SSH connection between chroot and DUT

To run automated tests on your [DUT](#), you first need to set up SSH connection between chroot on your workstation and the DUT. See [this document](#) for how to set it up.

For Googlers

If you are a Google engineer using a corp workstation, you may be required some extra settings, depending on where your DUT is.

- DUT at office: Use [corp-ssh-helper-helper](#).
- DUT at your home: See [go/arc-wfh](#).
- DUT at lab: See [go/chromeos-lab-duts-ssh](#).

Tast

[Tast](#) is a Go-based integration testing framework with a focus on speed, ease-of-use, and maintainability. Existing Autotest-based tests that run on the ChromeOS Commit Queue are being ported to Tast and decommissioned as of 2018 Q4. Please strongly consider using Tast when writing new integration tests (but be aware that not all functionality provided by Autotest is available in Tast; for example, tests that use multiple devices simultaneously when running are not currently supported).

Here are some starting points for learning more about Tast:

- [Tast Quickstart](#)
- [Tast: Running Tests](#)
- [Tast: Writing Tests](#)
- [Tast Overview](#)

Please contact the public [fast-users mailing list](#) if you have questions.

Autotest

[Autotest](#) is a Python-based integration testing framework; the codebase is also responsible for managing the [ChromeOS lab](#) that is used for hardware testing. ChromiumOS-specific Autotest information is available in the [Autotest User Documentation](#).

Additional Autotest documentation:

- [Creating a new Autotest test](#)
- [Running Autotest Smoke Suite On a VM Image](#)
- [Seeing which Autotest tests are implemented by an ebuild](#)

Creating a normal image that has been modified for test

See [Creating an image that has been modified for test](#) for information about modifying a normal system image so that integration tests can be run on it.

Creating a recovery image that has been modified for test

After building a test image using `cros build-image test` as described above, you may wish to encapsulate it within a recovery image:

```
(outside)
$ cros_sdk ./mod_image_for_recovery.sh ¥
  --board=${BOARD} ¥
  --nominimize_image ¥
  --image ~/chromiumos/src/build/images/${BOARD}/latest/chromiumos_test_image.bin ¥
  --to ~/chromiumos/src/build/images/${BOARD}/latest/recovery_test_image.bin
```

If desired, you may specify a custom kernel with `--kernel_image ${RECOVERY_KERNEL}`.

You can write this recovery image out to the USB device like so:

```
(outside)
$ cros flash usb:// ~/chromiumos/src/build/images/${BOARD}/latest/recovery_test_image.bin
```

Note that there are some **downsides** to this approach which you should keep in mind.

- Your USB image will be a recovery mode/test image, but the ordinary image in your directory will be a non-test image.
- If you use devserver, this will serve the non-test image not the test-image.
- This means a machine installed with a test-enabled USB image will update to a non-test-enabled one.
- As the `*-generic` boards set `USE=tpm`, recovery images built for `*-generic` don't work on devices with H1 chips (which requires `USE=tpm2`).

Additional information

Entering a shell in the chroot

For commands which need executed in the SDK, we've previously prefixed all them with `cros_sdk` in this guide. However, you may find that getting a shell in the chroot is useful for debugging or some other advanced workflow. You can do so by running `cros_sdk` without any arguments:

```
(outside)
$ cros_sdk
```

This command will probably prompt you for your password for the `sudo` command (entering the chroot requires root privileges). Once the command finishes, that terminal is in the chroot and you'll be in the `~/chromiumos/src/scripts` directory, where most build commands live. In the chroot you can only see a subset of the filesystem on your machine. However, through some trickery (bind mounts), you will have access to the whole `src` directory from within the chroot – this is so that you can build the software within the chroot.

Note in particular that the `src/scripts` directory is the same `src/scripts` directory found within the ChromiumOS directory you were in before you entered the chroot, even though it looks like a different location. That's because when you enter the chroot, the `~/chromiumos` directory in the chroot is mounted such that it points to the main ChromiumOS directory `~/chromiumos` outside the chroot. That means that changes that you make to the source code outside of the chroot immediately take effect inside the chroot.

Calling this will also install a chroot, if you don't have one yet, for example by not following the above.

While in the chroot you will see a special "(cr)" prompt to remind you that you are there:

```
(cr) ((...)) johnnyrotten@flyingkite ~/chromiumos/src/scripts $
```

You generally cannot run programs on your filesystem from within the chroot. For example, if you are using eclipse as an IDE, or gedit to edit a text file, you will need to run those programs outside the chroot. For lightweight editing, `vi` (not `vim`) and `nano` are available.

SIDE NOTES:

- If you need to share lots of files inside and outside chroot (for example, settings for your favorite editor / tools, or files downloaded by browser outside chroot, etc.), read [Tips and Tricks](#).
- There is a file system loop because inside `~/chromiumos` you will find the chroot again. Don't think about this for too long. If you try to use `du -s ~/chromiumos/chroot/home` you might get a message about a corrupted file system. This is nothing to worry about, and just means that your computer doesn't understand this loop either. (If you *can* understand this loop, try [something harder](#).)

Inspecting a disk image

To inspect a disk image, such as one built with the `cros build-image` command as in section [Build a disk image for your board](#), first launch an interactive Bash shell within the SDK chroot:

```
(outside)
$ cros_sdk
```

Then, mount the image and inspect the `/tmp/m` directory (here [...] denotes parts of the output omitted for brevity; (inside) denotes any commands executed within the SDK Bash shell):

```
(inside)
$ ./mount_gpt_image.sh --board=${BOARD} --safe --most_recent
[...]
[...] Image specified by [...] mounted at /tmp/m successfully.
```

If you built a test image, also make sure to add `-i chromiumos_test_image.bin` to this command.

The `--safe` option ensures you do not make accidental changes to the Root FS.

When you're done, unmount the image with:

```
(inside)
$ ./mount_gpt_image.sh --board=${BOARD} -u
```

Then `exit` the SDK chroot Bash shell.

Optionally, you can unpack the partition as separate files and mount them directly:

```
(outside)
$ cd ~/chromiumos/src/build/images/${BOARD}/latest
$ ./unpack_partitions.sh chromiumos_image.bin
$ mkdir -p rootfs
$ sudo mount -o loop,ro part_3 rootfs
```

This will do a loopback mount of the rootfs from your image to the location `~/chromiumos/src/build/images/${BOARD}/latest/rootfs` in your chroot.

If you built with `--no-enable-rootfs-verification` you can omit the `ro` option to mount it read write.

If you built an x86 ChromiumOS image, you can probably even try chrooting into the image:

```
(outside)
$ sudo chroot ~/chromiumos/src/build/images/${BOARD}/latest/rootfs
```

This is a little hacky (the ChromiumOS rootfs isn't really designed to be a chroot for your host machine), but it seems to work pretty well. Don't forget to `exit` this chroot when you're done.

When you're done, unmount the root filesystem:

```
(outside)
$ sudo umount ~/chromiumos/src/build/images/${BOARD}/latest/rootfs
```

Working on SDK Packages

SDK packages are normally automatically updated from tarball when you enter the SDK, and there's generally no need to manually update SDK packages yourself, unless you're actually working on a change which makes a change to the SDK.

When making a change to the SDK, you must know that your change won't be generally deployed until the next tarball is generated, roughly every 12 hours. Thus, if your SDK change is needed for building board packages, you should separate the change into two CLs, and land the change which incorporates the SDK package change after the tarball uprevs.

If the package you're working on is a `cros-workon` package, you'll want to workon the package first:

```
(outside)
$ cros workon --host chromeos-base/my-package
```

Then, to test your SDK package changes locally, you can manually emerge the package:

```
(outside)
$ cros_sdk sudo emerge -guj --deep --newuse chromeos-base/my-package
```

Alternatively, you may update all the SDK packages, including your package:

```
(outside)
$ cros_sdk update_chroot --force
```

`--force` is required to say, "yes, I really want to update my SDK from the current pinned version." Historically, developers may have been used to calling `update_chroot` manually, even if they didn't have SDK package changes they made that they want to incorporate, and we added `--force` to train developers that they generally shouldn't need to call this command manually.

Documentation on this site

You now understand the basics of building, running, modifying, and testing ChromiumOS, but you've still got a lot to learn. Here are links to a few other pages on the chromium.org site that you are likely to find helpful (somewhat ordered by relevance):

- The [Tips And Tricks](#) page has a lot of useful information to help you optimize your workflow.
- If you haven't read the page about the [devserver](#) already, read it now.
- Learn about the ChromiumOS [directory structure](#).
- [The ChromiumOS developer FAQ](#)
- [ChromiumOS Portage Build FAQ](#) is useful for portage-specific questions.
- If you have questions about the `--no-enable-rootfs-verification` option, you might find answers on this [thread on chromium-os-dev](#).
- [Running Smoke Suite on a VM Image](#) has good information about how to get up and running with autotest tests and VMs.
- [Debugging Tips](#) contains information about how to debug the Chromium browser on your ChromiumOS device.
- [Working on a Branch](#) has tips for working on branches.
- [Git server-side information](#) talks about adding new git repositories.
- The [Portage Package Upgrade Process](#) documents how our Portage packages can be upgraded when they fall out of date with respect to upstream.
- The [ChromiumOS Sandboxing](#) page describes the mechanisms and tools used to sandbox (or otherwise restrict the privileges of) ChromiumOS system services.
- The [Go in ChromiumOS](#) page provides information on using [Go](#) in ChromiumOS, including recommendations for project organization, importing and managing third party packages, and writing ebuids.
- The [SELinux](#) page provides information on SELinux in ChromeOS, including overview, writing policies, and troubleshooting.
- The [Running a single binary with UBSAN](#) page gives tips for using UBSAN (undefined behavior sanitizer) to find bugs in a binary. Instructions can also be used for ASAN (address sanitizer) and TSAN (thread sanitizer).

External documentation

Below are a few links to external sites that you might also find helpful (somewhat ordered by relevance):

- **Definitely** look at the [ChromiumOS dev group](#) to see what people are talking about.
- Check out the [Chromium bug tracker](#) to report bugs, look for known issues, or look for things to fix.

- Get an idea of what's actively happening by looking at the [Chromium Gerrit](#) site. (Note that this is no longer the same site that Chromium uses)
- Browse the source code on the [ChromiumOS gitweb](#).
- Check the current status of the builds by looking at the [ChromiumOS build waterfall](#).
- Check out the #chromium-os channel on [libera.chat](#) (some ChromiumOS developers hang out on this IRC channel).
- If you're learning git, check out [Git for Computer Scientists](#), [Git Magic](#), or the [Git Manual](#).
- Gentoo has several useful documentation resources
 - [Gentoo Development Guide](#) is a useful resource for Portage, which is the underlying packaging system we use.
 - [Gentoo Embedded Handbook](#)
 - [Gentoo Cross Development Guide](#)
 - [Gentoo Wiki on Cross-Compiling](#)
 - [Gentoo Package Manager Specification](#)
- The [repo user docs](#) might help you if you're curious about repo.
- The [repo-discuss group](#) is a good place to talk about repo.